МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Чувашский государственный университет имени И.Н. Ульянова»

Факультет информатики и вычислительной техники

Кафедра вычислительной техники

ТЕОРИЯ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ И МЕТОДЫ ТРАНСЛЯЦИИ

Расчетно-графическая работа (часть 2)

Синтаксически управляемая трансляция

Выполнил:
студент группы ИВТ-42-23
Иванов Иван

Руководитель:
доцент Павлов Л.А.

Чебоксары 2026

Оглавление

1. Контекстно-свободная грамматика языка.....5
2. Синтаксически управляемое определение и схема трансляции.....11
2.1. Семантический анализ (проверка типов).....11
2.2. Генерация промежуточного кода.....12
2.3. Разработка СУО.....12
2.4. Разработка СУТ.....13
3. Структуры данных и алгоритмы.....24
4. Программная реализация СУ-трансляции.....26
Заключение.....27
Список использованной литературы.....28

Задание к РГР

1. Проработать теоретические вопросы, связанные с восходящей синтаксически управляемой трансляцией языка варианта 8 (package8 IvanovIL), разработанного в 1-й части РГР: разработка СУО и СУТ для реализации семантического анализа (проверки типов) и генерации промежуточного кода.

2. Разработать программный модуль восходящей СУ-трансляции: структуры данных таблицы разбора и таблицы символов, процедуры семантических действий, генератор трехадресного кода.

Введение

Цель работы – изучение методов синтаксически управляемой трансляции (СУ-трансляция), включающей в себя фазы синтаксического и семантического анализа и генерации промежуточного кода; получение практических навыков построения моделей СУ-трансляции; разработка СУ-транслятора в соответствии с заданным вариантом.

Основой СУ-трансляции является включение *семантических действий* в процесс синтаксического анализа. Часто применяемыми моделями СУ-трансляции являются синтаксически управляемые определения и синтаксически управляемые схемы трансляции.

Синтаксически управляемое определение (СУО) представляет собой контекстно-свободную грамматику, дополненную атрибутами и семантическими правилами.

Для нетерминалов выделяют два типа атрибутов: синтезируемые и наследуемые.

Терминал может иметь только синтезируемый атрибут, который является атрибутом токена, передаваемого лексическим анализатором.

Синтаксически управляемая схема трансляции (СУТ) представляет собой контекстно-свободную грамматику, дополненную программными фрагментами (*семантическими действиями*), вставленными в правые части продукций. Отличие СУТ от СУО заключается в том, что в СУТ явно определен порядок вычисления семантических правил, задаваемый порядком обхода дерева разбора, который, в свою очередь, определяется используемым методом синтаксического анализа.

В процессе разработки СУТ обычно приходится решать следующие задачи:

- выделить конструкции заданного языка, обрабатываемые на этапе синтаксического анализа, и определять для них соответствующую контекстно-свободную грамматику;
- определить принадлежность контекстно-свободной грамматики к классу *LL*(1)- или *LR*(1)-грамматик;
- при необходимости выполнить эквивалентные преобразования грамматики в *LL*(1)- или *LR*(1)-форму;
- построить синтаксически управляемую схему трансляции, включив в продукции грамматики семантические правила, реализующие проверку типов и генерацию промежуточного кода;
- построить таблицу разбора для реализации синтаксически управляемой трансляции;
- разработать структуру СУ-транслятора, определить его функции и алгоритмы и программно его реализовать.

1. Контекстно-свободная грамматика языка

Язык варианта 8 — строго типизированный императивный язык. Программа начинается с ключевых слов package8 IvanovIL, затем следуют раздел объявлений (typedef8 — объявление типов, var8 — объявление переменных) и раздел операторов, заключённый в фигурные скобки.

Базовые типы языка: integer (4 байта), float (8 байт) и logical (4 байта). Составной тип struct8 позволяет объявлять запись, поля которой могут иметь как базовые, так и ранее объявленные составные типы; доступ к полю выполняется через точку. Допустимы оператор присваивания = и управляющий оператор do8 … if8 (Выражение) exit8 enddo8. Выражения строятся из констант, переменных, обращений к полям, унарных +, –, not и бинарных операций +, –, *, /, сравнений, & и |.

Множество лексических классов языка приведено в таблице 1.

Таблица 1. Лексические классы языка

№	Токен	Имя	Код токена	Формальный шаблон	Значение атрибута
1	package8	**pckg**	lcPckg	package8	0
2	IvanovIL	**name**	lcIvanovIL	IvanovIL	указатель на запись в таблице символов
3	{	**{**	lcStart	{	0
4	}	**}**	lcStop	}	0
5	var8	**var**	lcVar	var8	0
6	do8	**do**	lcDo	do8	0
7	if8	**if**	lcIf	if8	0
8	exit8	**exit**	lcExit	exit8	0
9	enddo8	**enddo**	lcEnddo	enddo8	0
10	typedef8	**typedef**	lcTypedef	typedef8	0
11	struct8	**struct**	lcStruct	struct8	0
12	точка с запятой	**;**	lcSemi	;	0
13	точка	**.**	lcDot	.	0
14	откр. скобка	**(**	lcOpPar	(	0
15	закр. скобка	**)**	lcClPar	)	0
16	присваивание	**ass**	lcAss	=	0
17	присваивание типа	**typeass**	lcTypeAss	<-	0
18	операция сравнения	**comp**	lcComp	< | <= | > | >= | == | !=	opL (<) opLe (<=) opG (>) opGe (>=) opE (==) opNe (!=)
19	логичИ	**and**	lcAnd	&	0
20	логичИЛИ	**or**	lcOr	|	0
21	логичНЕ	**not**	lcNot	!	0
22	логичИСТИНА	**true**	lcTrue	true	0
23	логичЛОЖЬ	**false**	lcFalse	false	0
24	аддитивная операция	+	lcAdd	+ | –	opAdd (+) opSub (–)

25	мультипл. операция	*	lcMult	* | /	opMult (*) opDiv (/)
26	идентификатор	**id**	lcId	l (l | d)* l – буква, d – цифра	указатель на запись в таблице символов
27	число	**num**	lcNum	d+ | d+.d+ d – цифра	указатель на запись в таблице символов
28	пробел	**sp**		(sp | tab | lf)+ sp – пробел, tab – табуляция, lf – перевод строки	токен не формируется
29	комментарий	**com**		<*X*> X – любой символ кроме *>	токен не формируется
30	маркер конца ввода	**eof**	lcEof		0
31	лексическая ошибка	**err**	lcErr		0

Синтаксис языка в форме РБНФ имеет следующий вид:

РБНФ

- Программа = “**package8**” “IvanovIL” "**;**" РазделОбъявлений РазделОператоров.
- РазделОбъявлений = [РазделТипов] РазделПеременных
- РазделТипов = "**typedef8**" ОпределениеТипа "**;**" { ОпределениеТипа "**;**"}
- ОпределениеТипа = Имя_типа “<-“ Тип
- РазделПеременных = "**var8**" ОбъявлениеПеременной "**;**" { ОбъявлениеПеременной "**;**"}
- ОбъявлениеПеременной = Тип Имя_переменной
- Тип = Имя_Типа | СоставнойТип**.**
- Имя = Буква { Буква | Цифра }**.**
- СоставнойТип = "**struct8**" "{" Тип Имя_поля "**;**" { Тип Имя_поля "**;**"} "}"
- РазделОператоров = "{" ПоследОператоров "}"**.**
- ПоследОператоров = Оператор "**;**" { Оператор "**;**"} **.**
- Оператор = ОператорПрисваивания | УправляющийОператор**.**
- ОператорПрисваивания = Переменная “=” Выражение**.**
- Переменная = Имя_Переменной | ОбращениеКЭлементуСоставногоТипа**.**
- ОбращениеКЭлементуСоставногоТипа = Переменная "**.**" Имя_поля
- УправляющийОператор ="**do8**" ПоследОператоров "**if8**" "**(**" Выражение "**)**" "**exit8**" "**enddo8**"
- Выражение = ПростоеВыраж [ОперОтношения ПростоеВыраж]**.**
- ОперОтношения = "<" | "<=" | ">" | ">=" | “==”| “!=”**.**
- ПростоеВыраж = ["+" | "–"] Терм { "+" | "–" Терм } | Терм { “|” Терм }**.**
- Терм = Фактор { "*" | "/" Фактор } | Фактор { “&” Фактор }**.**
- Фактор = Целое ["**.**" Целое] | "**true**" | "**false**" | Переменная | "**(**" Выражение "**)**" | “!” Фактор**.**
- Целое = Цифра { Цифра }**.**

Выполнив замену конструкций РБНФ, построим формальную контекстно-свободную грамматику G = (VT, VN, P, S), где VT — множество терминалов, VN — множество нетерминалов, P — множество продукций, S = Prog — начальный символ.

ФОРМАЛЬНАЯ ГРАММАТИКА ЯЗЫКА

Prog → **pckg name ;** DeclPart StmtPart.
DeclPart → TypePart VarPart | VarPart.
TypePart → **typedef** LstTypeDef.
LstTypeDef → TypeDef **;** | LstTypeDef TypeDef **;**.
TypeDef → **id typeass** Type.
VarPart → **var** LstVarDecl.
LstVarDecl → VarDecl **;** | LstVarDecl VarDecl **;**.
VarDecl → Type **id**.
Type → **id** | StructType.
StructType → **struct {** LstField **}**.
LstField → Field **;** | LstField Field **;**.

Field → Type **id**.
StmtPart → **{** LstStmt **}**.
LstStmt → Stmt **;** | LstStmt Stmt **;**.
Stmt → AssignStmt | CtrlStmt.
AssignStmt → Var **ass** Expr.
Var → **id** | StructAccess.
StructAccess → Var **. id**.
CtrlStmt → **do** LstStmt **if (** Expr **) exit enddo**.
Expr → SimpleExpr | SimpleExpr **comp** SimpleExpr.
SimpleExpr → MathExpr | OrExpr.
MathExpr → Term | **add** Term | MathExpr **add** Term.
OrExpr → Term **or** Term | OrExpr **or** Term.
Term → MultTerm | AndTerm.
MultTerm → Factor | MultTerm **mult** Factor.
AndTerm → Factor **and** Factor | AndTerm **and** Factor.
Factor → **num** | **true** | **false** | Var | **(** Expr **)** | **not** Factor.

Начальный символ Prog дополнен терминалом eof, который обозначает конец входного потока. Полученная грамматика является SLR(1)-грамматикой: для неё генератор таблиц построил 101 состояние и 61 столбец без конфликтов, что достаточно для восходящего анализа с однозначным выбором действия по таблице разбора.

Для реализации синтаксически управляемой трансляции грамматика расширена маркерами M1–M7 и символами действий A2–A32; расширенная грамматика приведена в разделе 2.4.

2. Синтаксически управляемое определение и схема трансляции

2.1. Семантический анализ (проверка типов)

Семантическими соглашениями языка являются следующие.

1. Никакой идентификатор не может быть объявлен в программе более одного раза; повторное объявление фиксируется ошибкой 1.

2. Каждому использующему вхождению идентификатора должно предшествовать объявление; иначе фиксируется ошибка 5.

3. Язык строго типизирован: неявные преобразования типов отсутствуют. Эквивалентность типов структурная — два типа эквивалентны, если совпадают их код и (для записей) таблица полей.

4. Предопределённые типы: integer (4 байта), float (8 байт), logical (4 байта). Для типа-записи размер равен сумме размеров полей с учётом смещений; адрес размещения объекта кратен размеру типа.

5. В определении типа typedef8 идентификатор слева от <- может быть только новым именем типа; в объявлении переменной var8 и в операторе присваивания идентификатор должен быть именем переменной.

6. Операнды арифметических операций +, –, *, / должны быть числовыми (integer или float) и иметь одинаковый тип; операнды логических операций &, |, not — типа logical; условие цикла — типа logical.

7. Операнды сравнений должны иметь одинаковый тип; для logical допустимы только == и != , для числовых — все операции сравнения.

8. Обращение к полю допустимо только для переменной типа-записи, а имя поля должно присутствовать в таблице полей этой записи.

В таблице символов идентификатор может иметь одну из категорий: catNoCat (категория ещё не определена), catProgName (имя программы), catTypeName (имя типа), catVarName (имя переменной), catConst (константа). Для типа-записи в элементе таблицы хранится ссылка на отдельную таблицу полей.

Таблица 2. Ошибки, обнаруживаемые при СУ-трансляции

Код	Название	Причина
1	Повторное объявление	Идентификатор уже объявлен в программе
2	Не имя типа	В позиции имени типа указан не тип
3	Не имя переменной	В позиции переменной указан не объект
4	Несовместимость типов	Типы операндов или операнда и результата не совпадают
5	Не объявлен	Идентификатор не описан в таблице символов
6	Не числовой операнд	Операнд арифметики имеет тип logical или void
7	Не логический операнд	Операнд логической операции или условия не logical

8	Не запись	Обращение к полю выполнено не у переменной-записи
9	Поле не найдено	Поля с таким именем нет в таблице полей записи

Проверка типов выполняется семантическими действиями в моменты свёртки продукций. Для арифметических выражений тип результата совпадает с типом операндов; для сравнений и логических операций результат имеет тип logical. Для оператора присваивания типы левой и правой частей должны быть эквивалентны. Для цикла проверяется, что условие имеет тип logical, а тело транслируется в трехадресный код с условным и безусловным переходами.

2.2. Генерация промежуточного кода

Промежуточный код представляется последовательностью трехадресных команд (четвёрок) вида (op, arg1, arg2, res), где op — код операции, arg1 и arg2 — операнды (индексы элементов таблицы символов), res — результат. Для временных значений используются переменные t1, t2, …; константы и переменные хранятся в таблице символов и имеют адреса в памяти данных.

Перечень используемых трехадресных команд приведён в таблице 3.

Таблица 3. Трехадресные команды

Команда	Обозначение	Назначение
opAss	=	Присваивание res := arg1
opAdd / opSub	+ / –	Сложение и вычитание
opMult / opDiv	* / /	Умножение и деление
opOr / opAnd / opNot	or / and / not	Логические операции
opL / opLe / opG / opGe	< / <= / > / >=	Сравнения числовых операндов
opE / opNe	== / !=	Сравнение на равенство и неравенство
opIfGoto	if goto	Условный переход при истинности arg1
opGoto	goto	Безусловный переход

Логические выражения и сравнения вычисляются в временные переменные типа logical, после чего управляющий оператор использует условный переход. Например, для фрагмента

```
a = 10 + 20 * 30;
b = a / 6 - 4;
```

будет сгенерирована последовательность:

```
t1 = 20 * 30
t2 = 10 + t1
a  = t2
t3 = a / 6
t4 = t3 - 4
b  = t4
```

Для цикла do8 … exit8 enddo8 в начале тела запоминается номер текущей команды, а в конце генерируются условный переход на команду после цикла и безусловный переход на начало тела. Это соответствует методу обратных поправок и обеспечивает однопроходную генерацию кода.

2.3. Разработка СУО

Для реализации трансляции нетерминалы грамматики наделены атрибутами. Синтезируемые атрибуты вычисляются после разбора правой части продукции, наследуемые — до разбора. В работе используются следующие атрибуты: type (код типа, размер и ссылка на таблицу полей),

addr (адрес в памяти данных или индекс элемента таблицы символов), pnt (индекс идентификатора в таблице символов), op (код операции) и instr (номер команды или метки).

Атрибут type имеет структуру (TypeCode, Width, Fields): TypeCode — код типа (typeVoid, typeInt, typeFloat, typeLogical, typeRecord), Width — размер в байтах, Fields — индекс таблицы полей для типа-записи. Для базовых типов Fields = –1.

Таблица символов содержит элементы со следующими полями: Lex (лексема), Cat (категория), TypeCode (код типа), Width (размер), Addr (адрес), Fields (ссылка на таблицу полей). Для записей создаются отдельные таблицы полей, в которых хранятся имя поля, его тип, размер и смещение внутри записи.

Основные процедуры и функции СУО:

— AddType(pnt, cat, type) — заносит в таблицу символов категорию и тип идентификатора; для переменной выделяет память в области данных;

— AddField(table, lex, type, offset) — добавляет поле в таблицу полей записи с заданным смещением;

— NewTbl() — создаёт новую пустую таблицу полей;

— Record(table) — строит тип-запись, ссылающийся на таблицу полей;

— FindField(table, lex) — ищет поле по имени в таблице полей;

— NewTemp(type) — создаёт временную переменную и выделяет ей память;

— ConstZero(type), ConstBool(value) — возвращают элементы таблицы для констант 0 и true/false;

— EnsureAddr(pnt) — выделяет адрес константе или литералу при первом использовании;

— Alloc(width) — выделяет память с выравниванием по размеру типа;

— Gen(op, arg1, arg2, res) — добавляет трехадресную команду.

Алгоритм процедуры AddType:

```
AddType(pnt, cat, type):
  if pnt.Cat != catNoCat then Type_Error(1)
  pnt.Cat := cat
  pnt.TypeCode := type.Code
  pnt.Width := type.Width
  pnt.Fields := type.Fields
  if cat = catVarName then
     pnt.Addr := Alloc(type.Width)
```

При обработке записи действие A32 создаёт новую таблицу полей и запоминает текущее смещение в стеке SO, после чего смещение обнуляется. Действие A9 добавляет очередное поле в таблицу с текущим смещением и увеличивает смещение на размер поля. Действие A6 извлекает таблицу полей из стека ST, строит тип Record и восстанавливает предыдущее смещение. Такой подход соответствует L-атрибутной схеме и позволяет обрабатывать вложенные записи.

2.4. Разработка СУТ

СУТ получается включением в продукции грамматики символов действий. Действия, которые должны выполняться в середине продукции, вынесены в маркеры M1–M7 с пустыми правыми

частями; действия в конце продукции записываются непосредственно после правой части. Нумерация действий соответствует таблице СУО и СУТ.

При восходящем разборе к моменту выполнения действия текущий входной токен уже прочитан, поэтому для доступа к атрибутам терминала (id.pnt, num.pnt, код операции) используется переменная PredToken, хранящая последний прочитанный терминал.

Грамматика с действиями, подготовленная для генератора таблиц, имеет следующий вид:

ДЛЯ ГЕНЕРАТОРА

```
Prog = "pckg" "name" ";" DeclPart StmtPart "eof".
DeclPart = TypePart VarPart | VarPart.
TypePart = "typedef" LstTypeDef.
LstTypeDef = TypeDef ";" | LstTypeDef TypeDef ";".
TypeDef = "id" M2 "typeass" Type {A3}.
VarPart = "var" LstVarDecl.
LstVarDecl = VarDecl ";" | LstVarDecl VarDecl ";".
VarDecl = Type "id" {A4}.
Type = "id" {A5} | StructType.
StructType = "struct" M1 "{" LstField "}" {A6}.
LstField = Field ";" | LstField Field ";".
Field = Type "id" {A9}.
StmtPart = "{" LstStmt "}".
LstStmt = Stmt ";" | LstStmt Stmt ";".
Stmt = AssignStmt | CtrlStmt.
AssignStmt = Var "ass" Expr {A11}.
Var = "id" {A12} | StructAccess.
StructAccess = Var "." "id" {A22}.
CtrlStmt = "do" M3 LstStmt "if" "(" Expr ")" "exit" "enddo" {A24}.
Expr = SimpleExpr | SimpleExpr "comp" M4 SimpleExpr {A26}.
SimpleExpr = MathExpr | OrExpr.
MathExpr = Term | "add" M5 Term {A14} | MathExpr "add" M6 Term {A16}.
OrExpr = Term "or" Term {A27} | OrExpr "or" Term {A28}.
Term = MultTerm | AndTerm.
MultTerm = Factor | MultTerm "mult" M7 Factor {A18}.
AndTerm = Factor "and" Factor {A29} | AndTerm "and" Factor {A30}.
M1 = {A32}.
M2 = {A2}.
M3 = {A23}.
M4 = {A25}.
M5 = {A13}.
M6 = {A15}.
M7 = {A17}.
Factor = "num" {A19} | "true" {A20} | "false" {A21} | Var | "(" Expr ")" | "not" Factor
{A31}.!
```

По этой грамматике генератор построил SLR(1)-таблицу разбора. Фрагмент таблицы СУО и СУТ, перенесённый из рабочего файла без изменения форматирования, приведён ниже.

Таблица 4. СУО и СУТ для проверки типов и трансляции в трехадресный код

Продукция с действиями	**Семантические правила**	**Семантические действия**	**Символ действия**
Prog → **pckg name ;** DeclPart StmtPart			
DeclPart → TypePart VarPart | VarPart			
TypePart → **typedef** LstTypeDef			
LstTypeDef → TypeDef **;** | LstTypeDef1 TypeDef **;**			
TypeDef → **id** {A2} **typeass** Type {A3}	**if id.** *pnt.Cat* ≠ *catNoCat* **then** *Type_Error*(1)	**if id.***pnt.Cat* ≠ *catNoCat* **then** *Type_Error*(1) *Push*(**id.***pnt*, *AttrSt*) // id.pnt	A2

	AddType(**id.***pnt*, *catTypeName*, Type.*type*)	*t1* := *Pop*(*AttrSt*) // Type.type
		t2 := *Pop*(*AttrSt*) // id.pnt
		AddType(*t2*, *catTypeName*, *t1*)
			A3

VarPart → **var** LstVarDecl			
LstVarDecl → VarDecl **;** | LstVarDecl1 VarDecl **;**			
VarDecl → Type **id** {A4}	**if id.***pnt.Cat* ≠ *catNoCat* **then** *Type_Error*(1) *AddType*(**id.** *pnt*, *catVarName*, Type.*type*)	**if id.***pnt.Cat* ≠ *catNoCat* **then** *Type_Error*(1) *t1* := *Pop*(*AttrSt*) // Type.type *AddType*(**id.***pnt*, *catVarName*, *t1*)	A4
Type → **id** {A5}	**if id.***pnt.Cat* ≠ *catTypeName* **then** *Type_Error*(2) Type.*type* := **id.***pnt.Type*	**if id.***pnt.Cat* ≠ *catTypeName* **then** *Type_Error*(2) *Push*(**id.***pnt.Type*, *AttrSt*)	A5
Type → StructType	Type.*type* := StructType.*type*	Правила копирования: в вершине стека уже находится атрибут StructType; действий со стеком не требуется	
StructType → **struct** {A32} **{** LstField **}** {A6}	*pNew* := *NewTbl*() *Push*(*pNew*, *ST*)	*t1* := *NewTbl*() *Push*(*t1*, *ST*)	A32

	Push(*offset*, *SO*) *offset* := 0	*Push*(*offset*, *SO*) *offset* := 0	
	p := *Pop*(*ST*) StructType.*type* := *Record*(*p*) *offset* := *Pop*(*SO*)	*t1* := *Pop*(*ST*) *Push*(*Record*(*t1*), *AttrSt*) // StructType.type *offset* := *Pop*(*SO*)	A6
LstField → Field **;**			
LstField → LstField1 Field **;**			
Field → Type **id** {A9}	**if** *FindField*(*Top*(*ST*), *id.pnt.Lex*) ≠ *nil* **then** *Type_Error*(1) *AddField*(*Top*(*ST*), *id.pnt.Lex*, Type.*type*)	*t1* := *Pop*(*AttrSt*) // Type.type **if** *FindField*(*Top*(*ST*), *id.pnt.Lex*) ≠ *nil* **then** *Type_Error*(1) *AddField*(*Top*(*ST*), *id.pnt.Lex*, *t1*)	A9
StmtPart → **{** LstStmt **}**			
LstStmt → Stmt **;** | LstStmt1 Stmt **;**			
Stmt → AssignStmt | CtrlStmt			

AssignStmt → Var **ass** Expr {A11}	**if** Var.*type* ≠ Expr.*type* **then** *Type_Error*(4)
	Gen(*opAss*, Expr.*addr*, -1, Var.*addr*)
		t1 := *Pop*(*AttrSt*) // Expr.addr
		t2 := *Pop*(*AttrSt*) // Expr.type
		t3 := *Pop*(*AttrSt*) // Var.addr
		t4 := *Pop*(*AttrSt*) // Var.type
		if *t4* ≠ *t2* **then** *Type_Error*(4)
		Gen(*opAss*, *t1*, -1, *t3*)
			A11

Var → **id** {A12}	**if** *id.pnt.Cat* = *catNoCat* **then** *Type_Error*(5) **else if** *id.pnt.Cat* ≠ *catVarName* **then** *Type_Error*(3) Var.*type* := *id.pnt*.Type Var.*addr* := *id.pnt*	**if** *id.pnt.Cat* = *catNoCat* **then** *Type_Error*(5) **else if** *id.pnt.Cat* ≠ *catVarName* **then** *Type_Error*(3) *Push*(*id.pnt*.Type, *AttrSt*) *Push*(*id.pnt*, *AttrSt*)	A12
Var → StructAccess	Var.*type* := StructAccess.*type* Var.*addr* := StructAccess.*addr*	Правила копирования: в вершине стека уже находятся атрибуты StructAccess; действий со стеком не требуется	

StructAccess → Var **. id**	**if** Var.*type.TypeCode* ≠ *typeRecord* **then**
	Type_Error(8)
	f := *FindField*(Var.*type.Fields*, *id.pnt.Lex*)
		t1 := *Pop*(*AttrSt*) // Var.addr
		t2 := *Pop*(*AttrSt*) // Var.type
		if *t2.TypeCode* ≠ *typeRecord* **then** *Type_Error*(8)
			A22

	if *f* = *nil* **then** *Type_Error*(9)
	StructAccess.*type* := *f*.Type
	StructAccess.*addr* := *FieldAddr*(Var.*addr*, *f*)
		t3 := *FindField*(*t2.Fields*, *id.pnt.Lex*)
		if *t3* = *nil* **then** *Type_Error*(9)
		Push(*t3*.Type, *AttrSt*) // StructAccess.type
		Push(*FieldAddr*(*t1*,*t3*), *AttrSt*) // StructAccess.addr	

CtrlStmt → **do** {A23} LstStmt **if (** Expr **) exit enddo** {A24}		*Push*(*nextinstr*, *AttrSt*) // M.instr	A23

	if Expr.*type.TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
	Gen(**if** Expr.*addr goto nextinstr* + 2)
	Gen(*goto M.instr*)
		t1 := *Pop*(*AttrSt*) // Expr.addr
		t2 := *Pop*(*AttrSt*) // Expr.type
		t3 := *Pop*(*AttrSt*) // M.instr
		if *t2.TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
		Gen(**if** *t1 goto nextinstr* + 2)
		Gen(*goto t3*)
			A24

Expr → SimpleExpr	Expr.*type* := SimpleExpr.*type* Expr.*addr* := SimpleExpr.*addr*	*Правила копирования*: *в вершине стека уже находятся атрибуты* SimpleExpr; *действий со стеком не требуется*	
Expr → SimpleExpr **comp** {A25} SimpleExpr {A26}		*Push*(*comp.op*, *AttrSt*) // атрибут comp.op	A25

	if *SimpleExpr1.type* ≠ *SimpleExpr2.type* **then** *Type_Error*(4)
	if *SimpleExpr1.type.TypeCode* = *typeLogical and comp.op* < *opE* **then** *Type_Error*(6)
	Expr.*type* := *TypeLogical*
	Expr.*addr* := *NewTemp*(*TypeLogical*)
	Gen(*comp.op*, *SimpleExpr1.addr*, *SimpleExpr2.addr*, Expr.*addr*)
		t1 := *Pop*(*AttrSt*) // SimpleExpr2.addr
		t2 := *Pop*(*AttrSt*) // SimpleExpr2.type
		t3 := *Pop*(*AttrSt*) // comp.op
		t4 := *Pop*(*AttrSt*) // SimpleExpr1.addr
		t5 := *Pop*(*AttrSt*) // SimpleExpr1.type
		if *t5* ≠ *t2* **then** *Type_Error*(4)
		if *t5.TypeCode* = *typeLogical and t3* < *opE* **then** *Type_Error*(6)
		Push(*TypeLogical*, *AttrSt*) // Expr.type
		t6 := *NewTemp*(*TypeLogical*)
		Push(*t6*, *AttrSt*) // Expr.addr
		Gen(*t3*, *t4*, *t1*, *t6*)
			A26

SimpleExpr → MathExpr	SimpleExpr.*type* := MathExpr.*type* SimpleExpr.*addr* := MathExpr.*addr*	*Правила копирования*: *в вершине стека уже находятся атрибуты* MathExpr; *действий со стеком не требуется*	
SimpleExpr → OrExpr	SimpleExpr.*type* := OrExpr.*type*	*Правила копирования*: *в вершине стека уже находятся*	

	SimpleExpr.*addr* := OrExpr.*addr*	*атрибуты* OrExpr; *действий со стеком не требуется*	
MathExpr → Term	MathExpr.*type* := Term.*type* MathExpr.*addr* := Term.*addr*	*Правила копирования*: *в вершине стека уже находятся атрибуты* Term; *действий со стеком не требуется*	
MathExpr → **add** {A13} Term {A14}		*Push*(*add.op*, *AttrSt*) // атрибут add.op	A13

	if Term.*type.TypeCode* = *typeLogical* **then** *Type_Error*(6)
	MathExpr.*type* := Term.*type*
	if *add.op* = *opSub* **then**
	MathExpr.*addr* := *NewTemp*(Term.*type*)
	Gen(*opSub*, *GetZero*(Term.*type*), Term.*addr*, MathExpr.*addr*)
	else MathExpr.*addr* := Term.*addr*
		t1 := *Pop*(*AttrSt*) // Term.addr
		t2 := *Pop*(*AttrSt*) // Term.type
		t3 := *Pop*(*AttrSt*) // add.op
		if *t2.TypeCode* = *typeLogical* **then** *Type_Error*(6)
		Push(*t2*, *AttrSt*) // MathExpr.type
		if *t3* = *opSub* **then**
		t4 := *NewTemp*(*t2*)
		Push(*t4*, *AttrSt*) // MathExpr.addr
		Gen(*opSub*, *GetZero*(*t2*), *t1*, *t4*)
		else Push(*t1*, *AttrSt*) // MathExpr.addr
			A14

MathExpr → MathExpr1 **add** {A15} Term {A16}		*Push*(*add.op*, *AttrSt*) // атрибут add.op	A15

	if MathExpr1.*type* ≠ Term.*type* **then** *Type_Error*(4)
	if MathExpr1.*type.TypeCode* = *typeLogical* **then** *Type_Error*(6)
	MathExpr.*type* := Term.*type*
	MathExpr.*addr* := *NewTemp*(Term.*type*)
	Gen(*add.op*, MathExpr1.*addr*, Term.*addr*, MathExpr.*addr*)
		t1 := *Pop*(*AttrSt*) // Term.addr
		t2 := *Pop*(*AttrSt*) // Term.type
		t3 := *Pop*(*AttrSt*) // add.op
		t4 := *Pop*(*AttrSt*) // MathExpr1.addr
		t5 := *Pop*(*AttrSt*) // MathExpr1.type
		if *t5* ≠ *t2* **then** *Type_Error*(4)
		if *t5.TypeCode* = *typeLogical* **then** *Type_Error*(6)
		Push(*t2*, *AttrSt*) // MathExpr.type
		t6 := *NewTemp*(*t2*)
		Push(*t6*, *AttrSt*) // MathExpr.addr
		Gen(*t3*, *t4*, *t1*, *t6*)
			A16

OrExpr → Term **or** Term	**if** Term1.*type* ≠ Term2.*type* **then** *Type_Error*(4)
	if Term1.*type.TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
	OrExpr.*type* := *TypeLogical*
	OrExpr.*addr* := *NewTemp*(*TypeLogical*)
	Gen(*opOr*, Term1.*addr*, Term2.*addr*, OrExpr.*addr*)
		t1 := *Pop*(*AttrSt*) // Term2.addr
		t2 := *Pop*(*AttrSt*) // Term2.type
		t3 := *Pop*(*AttrSt*) // Term1.addr
		t4 := *Pop*(*AttrSt*) // Term1.type
		if *t4* ≠ *t2* **then** *Type_Error*(4)
		if *t4.TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
			A27

		Push(*TypeLogical*, *AttrSt*) // OrExpr.type
		t5 := *NewTemp*(*TypeLogical*)
		Push(*t5*, *AttrSt*) // OrExpr.addr
		Gen(*opOr*, *t3*, *t1*, *t5*)	

OrExpr → OrExpr1 **or** Term	**if** OrExpr1.*type* ≠ Term.*type* **then** *Type_Error*(4)
	if OrExpr1.*type*.*TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
	OrExpr.*type* := *TypeLogical*
	OrExpr.*addr* := *NewTemp*(*TypeLogical*)
	Gen(*opOr*, OrExpr1.*addr*, Term.*addr*, OrExpr.*addr*)
		t1 := *Pop*(*AttrSt*) // Term.addr
		t2 := *Pop*(*AttrSt*) // Term.type
		t3 := *Pop*(*AttrSt*) // OrExpr1.addr
		t4 := *Pop*(*AttrSt*) // OrExpr1.type
		if *t4* ≠ *t2* **then** *Type_Error*(4)
		if *t4*.*TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
		Push(*TypeLogical*, *AttrSt*) // OrExpr.type
		t5 := *NewTemp*(*TypeLogical*)
		Push(*t5*, *AttrSt*) // OrExpr.addr
		Gen(*opOr*, *t3*, *t1*, *t5*)
			A28

Term → MultTerm	Term.*type* := MultTerm.*type* Term.*addr* := MultTerm.*addr*	*Правила копирования*: *в вершине стека уже находятся атрибуты* MultTerm; *действий со стеком не требуется*	
Term → AndTerm	Term.*type* := AndTerm.*type* Term.*addr* := AndTerm.*addr*	*Правила копирования*: *в вершине стека уже находятся атрибуты* AndTerm; *действий со стеком не требуется*	
MultTerm → Factor	MultTerm.*type* := Factor.*type* MultTerm.*addr* := Factor.*addr*	*Правила копирования*: *в вершине стека уже находятся атрибуты* Factor; *действий со стеком не требуется*	
MultTerm → MultTerm1 **mult** {A17} Factor {A18}		*Push*(*mult.op*, *AttrSt*) // атрибут mult.op	A17

	if MultTerm1.*type* ≠ Factor.*type* **then** *Type_Error*(4)
	if MultTerm1.*type*.*TypeCode* = *typeLogical* **then** *Type_Error*(6)
	MultTerm.*type* := Factor.*type*
	MultTerm.*addr* := *NewTemp*(Factor.*type*)
	Gen(*mult.op*, MultTerm1.*addr*, Factor.*addr*, MultTerm.*addr*)
		t1 := *Pop*(*AttrSt*) // Factor.addr
		t2 := *Pop*(*AttrSt*) // Factor.type
		t3 := *Pop*(*AttrSt*) // mult.op
		t4 := *Pop*(*AttrSt*) // MultTerm1.addr
		t5 := *Pop*(*AttrSt*) // MultTerm1.type
		if *t5* ≠ *t2* **then** *Type_Error*(4)
		if *t5*.*TypeCode* = *typeLogical* **then** *Type_Error*(6)
		Push(*t2*, *AttrSt*) // MultTerm.type
		t6 := *NewTemp*(*t2*)
			A18

		Push(*t6*, *AttrSt*) // MultTerm.addr *Gen*(*t3*, *t4*, *t1*, *t6*)	

AndTerm → Factor **and** Factor	**if** Factor1.*type* ≠ Factor2.*type* **then** *Type_Error*(4)
	if Factor1.*type.TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
	AndTerm.*type* := *TypeLogical*
	AndTerm.*addr* := *NewTemp*(*TypeLogical*)
	Gen(*opAnd*, Factor1.*addr*, Factor2.*addr*, AndTerm.*addr*)
		t1 := *Pop*(*AttrSt*) // Factor2.addr
		t2 := *Pop*(*AttrSt*) // Factor2.type
		t3 := *Pop*(*AttrSt*) // Factor1.addr
		t4 := *Pop*(*AttrSt*) // Factor1.type
		if *t4* ≠ *t2* **then** *Type_Error*(4)
		if *t4.TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
		Push(*TypeLogical*, *AttrSt*) // AndTerm.type
		t5 := *NewTemp*(*TypeLogical*)
		Push(*t5*, *AttrSt*) // AndTerm.addr
		Gen(*opAnd*, *t3*, *t1*, *t5*)
			A29

AndTerm → AndTerm1 **and** Factor	**if** AndTerm1.*type* ≠ Factor.*type* **then** *Type_Error*(4)
	if AndTerm1.*type.TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
	AndTerm.*type* := *TypeLogical*
	AndTerm.*addr* := *NewTemp*(*TypeLogical*)
	Gen(*opAnd*, AndTerm1.*addr*, Factor.*addr*, AndTerm.*addr*)
		t1 := *Pop*(*AttrSt*) // Factor.addr
		t2 := *Pop*(*AttrSt*) // Factor.type
		t3 := *Pop*(*AttrSt*) // AndTerm1.addr
		t4 := *Pop*(*AttrSt*) // AndTerm1.type
		if *t4* ≠ *t2* **then** *Type_Error*(4)
		if *t4.TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
		Push(*TypeLogical*, *AttrSt*) // AndTerm.type
		t5 := *NewTemp*(*TypeLogical*)
		Push(*t5*, *AttrSt*) // AndTerm.addr
		Gen(*opAnd*, *t3*, *t1*, *t5*)
			A30

Factor → **num**	Factor.*type* := *num.pnt*.Type Factor.*addr* := *num.pnt*	*Push*(*num.pnt*.Type, *AttrSt*) // Factor.type *Push*(*num.pnt*, *AttrSt*) // Factor.addr	A19
Factor → **true**	Factor.*type* := *TypeLogical* Factor.*addr* := *ConstTrue*	*Push*(*TypeLogical*, *AttrSt*) // Factor.type *Push*(*ConstTrue*, *AttrSt*) // Factor.addr	A20
Factor → **false**	Factor.*type* := *TypeLogical* Factor.*addr* := *ConstFalse*	*Push*(*TypeLogical*, *AttrSt*) // Factor.type *Push*(*ConstFalse*, *AttrSt*) // Factor.addr	A21
Factor → Var	Factor.*type* := Var.*type* Factor.*addr* := Var.*addr*	*Правила копирования*: *атрибуты* Var *уже в вершине стека*; *действий со стеком не требуется*	

Factor → **(**Expr**)**	Factor.*type* := Expr.*type* Factor.*addr* := Expr.*addr*	*Правила копирования*: *атрибуты* Expr *уже в вершине стека*; *скобки атрибутов не имеют*	

Factor → **not** Factor	**if** Factor1.*type*.*TypeCode* ≠ *typeLogical* **then**
	Type_Error(7)
	Factor.*type* := *TypeLogical*
	Factor.*addr* := *NewTemp*(*TypeLogical*)
	Gen(*opNot*, Factor1.*addr*, -1, Factor.*addr*)
		t1 := *Pop*(*AttrSt*) // Factor1.addr
		t2 := *Pop*(*AttrSt*) // Factor1.type
		if *t2*.*TypeCode* ≠ *typeLogical* **then** *Type_Error*(7)
		Push(*TypeLogical*, *AttrSt*) // Factor.type
		t3 := *NewTemp*(*TypeLogical*)
		Push(*t3*, *AttrSt*) // Factor.addr
		Gen(*opNot*, *t1*, -1, *t3*)
			A31

Действие A2 после идентификатора в определении типа проверяет, что идентификатор ещё не объявлен, и помещает его индекс в стек атрибутов. Действие A3 завершает определение типа и заносит в таблицу символов категорию catTypeName и тип правой части.

Действия A4 и A5 обрабатывают объявление переменной: A5 по имени типа возвращает его тип, а A4 проверяет идентификатор и заносит переменную в таблицу с выделением памяти. Действие A9 добавляет поле в текущую таблицу полей записи, а A32 создаёт эту таблицу перед разбором списка полей; A6 строит тип-запись и восстанавливает смещение.

Действие A11 проверяет совпадение типов левой и правой частей присваивания и генерирует команду opAss. Действие A22 ищет поле в таблице полей записи и вычисляет адрес поля как сумму адреса переменной и смещения поля. Действие A24 формирует условный переход по условию цикла и безусловный переход на начало тела, а A26 — сравнение операндов с проверкой их типов.

Действия A13, A15 и A17 запоминают код операции add или mult, а A14, A16 и A18 выполняют соответствующие арифметические операции и создают временные переменные. Действия A27–A30 реализуют логические операции or и and, A31 — операцию not; все они проверяют, что операнды имеют тип logical.

3. Структуры данных и алгоритмы

Таблица разбора LRTbl представляет собой прямоугольную матрицу, строки которой соответствуют состояниям анализатора, а столбцы — терминалам и нетерминалам грамматики. Символам действий отдельные столбцы не сопоставляются. Комбинация состояния в вершине стека анализатора и текущего входного символа является индексом таблицы, по которому определяется дальнейшее действие.

Элемент таблицы ElemLR содержит поля: ElType — тип элемента (lrErr = 0, lrShift = 1, lrReduct = 2, lrStop = 3); ElPar — параметр (для переноса — номер состояния, для свёртки — длина правой части); Left — код нетерминала левой части продукции; Act — код семантического действия (0 — действия нет). Для сопоставления токена и столбца таблицы используется массив LexCodeLR, заполняемый при загрузке таблицы из XML-файла.

Анализатор использует два стека: ParserSt — стек состояний и AttrSt — стек атрибутов. Начальное состояние 1 помещается в ParserSt. Текущий токен получается от сканера; переменная PredToken хранит предыдущий терминал, необходимый для выполнения действий при свёртке.

Алгоритм СУ-трансляции по таблице разбора:

```
ClearStack(AttrSt); ClearStack(ParserSt)
NextAddr := 0; NextInstr := 0
AddBaseTypes
Push(1, ParserSt)
Token := Scaner(); PredToken := Token
if Token.Code = lcErr then Lex_Error
Sym := LexCodeLR[Token.Code]
while ElemLR.ElType <> lrStop do
  State := StackTop(ParserSt)
  ElemLR := LRTbl[State, Sym]
  case ElemLR.ElType of
    lrErr:    Syntax_Error(State)
    lrShift:  Push(ElemLR.ElPar, ParserSt)
              if Sym – терминал then
                 PredToken := Token; Token := Scaner()
                 if Token.Code = lcErr then Lex_Error
                 Sym := LexCodeLR[Token.Code]
              else Sym := LexCodeLR[Token.Code]
    lrReduct: for i := 1 to ElemLR.ElPar do Pop(ParserSt)
              Sym := ElemLR.Left
              if ElemLR.Act <> 0 then
                 Semantic_Action(ElemLR.Act, PredToken)
  end
end
```

После загрузки таблицы из XML поле Left, содержащее имя нетерминала, переводится в номер столбца; для терминалов заполняется массив LexCodeLR. Начальное состояние анализатора — 1; элемент останова lrStop находится в строке, соответствующей допускающему состоянию, по токену eof.

При синтаксической ошибке анализатор выводит номер состояния и недопустимый токен, а также список терминалов, допустимых в этом состоянии. Список формируется по строке таблицы:

рассматриваются столбцы терминалов, элементы которых не являются элементами ошибки. Лексические ошибки обнаруживаются сканером, семантические — действиями A2–A32.

Для языка варианта 8 таблица содержит 101 состояние и 61 столбец (33 нетерминала и 28 терминалов). Размер таблицы фиксирован и не зависит от входной программы; при разборе используются только операции со стеком и обращения к таблице по индексам.

4. Программная реализация СУ-трансляции

Программный модуль СУТ реализован на C++17 с использованием библиотеки Qt 6 Widgets; для разбора XML-таблицы применена библиотека pugixml. Код разделён на модули, каждый из которых отвечает за свой этап трансляции.

Таблица 5. Модули программной реализации

Модуль	Назначение	Основные функции
LRTable	Загрузка и хранение таблицы разбора	Load, At, ColumnForToken, IsTerminal
SymbolTable	Таблица символов и таблицы полей	AddMain, AddField, NewTable, FindField, NewTemp, Alloc
Tac	Трехадресный код	Gen, GenIf, GenGoto, Quads
SemanticActions	Процедуры действий A2–A32	Execute, a2…a32
SutParser	LR-анализатор	LoadTable, Parse
MainWindow	Интерфейс	RunScanner, RunSut

Модуль LRTable читает XML-файл, формирует столбцы, строки и элементы таблицы, а также массив LexCodeLR. Модуль SymbolTable хранит основную таблицу символов, полученную от сканера, и отдельные таблицы полей записей; в нём реализованы выделение памяти, создание временных переменных и констант. Модуль Tac формирует последовательность четвёрок и переходы.

Модуль SemanticActions содержит процедуры всех действий. Каждая процедура получает доступ к стекам атрибутов и таблице символов, выполняет проверки типов и при необходимости генерирует команды. Модуль SutParser реализует алгоритм LR-разбора: хранит стек состояний, стек атрибутов, текущий и предыдущий токены, вызывает действия при свёртке.

В интерфейсе программы кнопка «СУТ» открывает диалог выбора файла LR_Table.xml, после чего выполняет разбор текста из редактора. Результаты отображаются на вкладках «Трехадресный код», «Таблица символов» и «Ошибки». Для сборки используются CMake и Makefile; таблица разбора загружается из файла, полученного генератором.

Проверка работоспособности выполнялась на тестовой программе, включающей объявление типов, записей, переменных, арифметические и логические выражения и цикл. Разбор завершился успешно, сгенерировано 38 трехадресных команд; для вложенных записей корректно вычислены адреса полей (l.a.x, l.b.y), а обращения к несуществующему полю и несовпадение типов дали ошибки 9 и 4 соответственно.

Заключение

В процессе выполнения расчетно-графической работы

- изучены методы синтаксически управляемой трансляции;
- получены практические навыки построения моделей СУ-трансляции (СУО и СУТ);
- разработана СУТ, ориентированная на восходящий разбор;
- разработаны структуры данных для представления таблиц СУ-транслятора;
- получены практические навыки разработки модуля СУ-трансляции.

В результате выполнения работы для языка варианта 8 построена SLR(1)-таблица разбора (101 состояние, 61 столбец), разработаны СУО и СУТ, реализованы структуры данных и программный модуль СУ-трансляции, выполнена проверка типов и генерация трехадресного кода.

Список использованной литературы

1. *Ахо, А.* Компиляторы: принципы, технологии и инструментарий / А. Ахо, М. Лам, Р. Сети, Д. Ульман. – 2-е изд.– М.: Вильямс, 2008. – 1184 с.

2. *Опалева, Э.А.* Языки программирования и методы трансляции / Э.А. Опалева, В.П. Самойленко.– СПб.: БХВ-Петербург, 2005.– 480 с.

3. *Павлов, Л.А.* Восходящий синтаксический анализ: конспект лекций / Л.А. Павлов.– Чебоксары: Изд-во Чуваш. ун-та, 2004. – 44 с.

4. *Павлов, Л.А.* Лексический и синтаксический анализ: учеб. пособие / Л.А. Павлов. – Чебоксары: Изд-во Чуваш. ун-та, 2022. – 152 с.

5. *Павлов, Л.А.* Нисходящий синтаксический анализ: конспект лекций / Л.А. Павлов.– Чебоксары: Изд-во Чуваш. ун-та, 2003. – 48 с.

6. *Павлов, Л.А.* Синтаксически управляемая трансляция: учеб. пособие / Л.А. Павлов. Чебоксары: Изд-во Чуваш. ун-та, 2017. – 60 с.

7. *Павлов, Л.А.* Структуры и алгоритмы обработки данных: учеб. пособие / Л.А. Павлов, Н.В. Первова. – 2-е изд. – Чебоксары: Изд-во Чуваш. ун-та, 2018. – 254 с.

8. *Павлов, Л.А.* Структуры и алгоритмы обработки данных: учебник / Л.А. Павлов, Н.В. Первова. – 3-е изд. – СПб.: Лань, 2021. – 256 с.

9. *Свердлов, С.З.* Языки программирования и методы трансляции: учеб. пособие / С.З. Свердлов.– СПб.: Питер, 2007.– 638 с.

10. *Свердлов, С.З.* Языки программирования и методы трансляции: учеб. пособие / С.З. Свердлов. – 2-е изд. – СПб.: Лань, 2019.– 564 с.

11. Теория языков программирования и методы трансляции: Метод. указания к выполнению расчетно-графических работ / Сост. Л.А. Павлов. – Чебоксары: Изд-во Чуваш. ун-та, 2015. – 48 с.